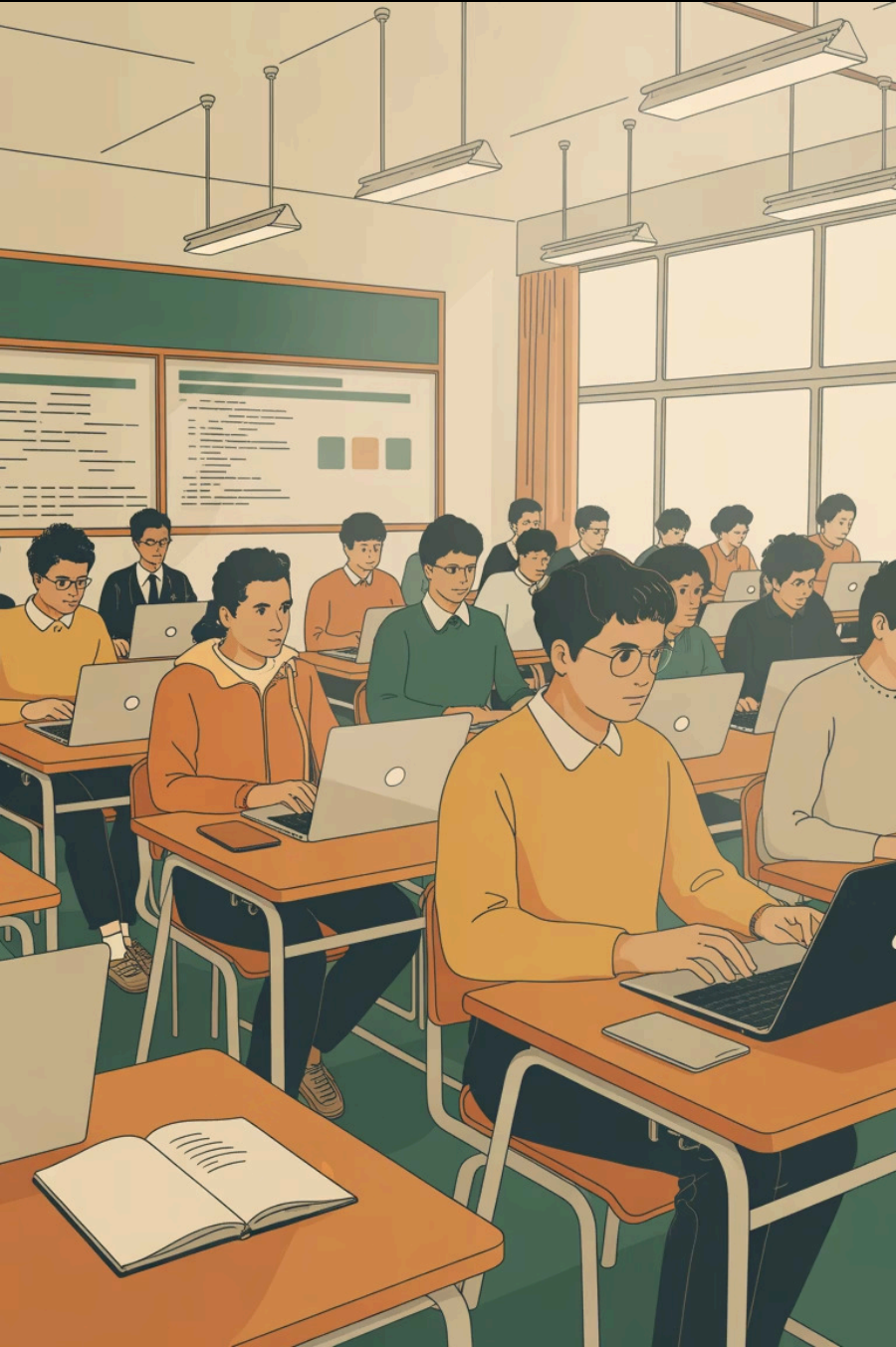




ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

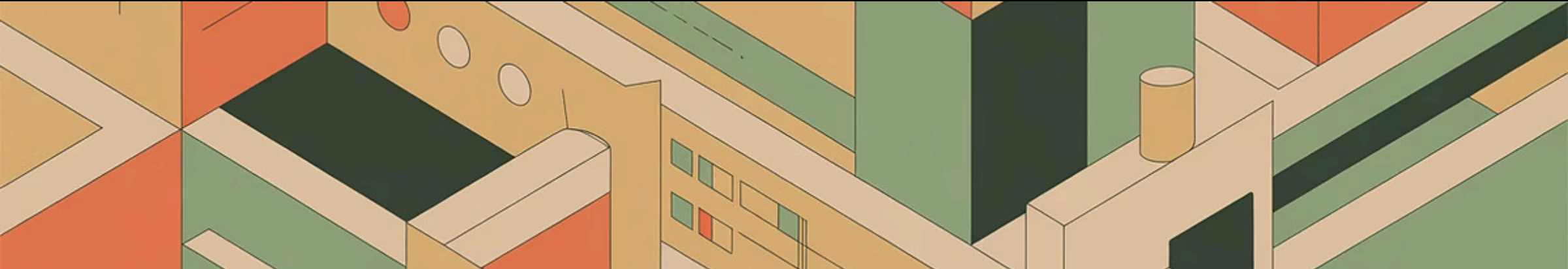
Requisitos para o Desenvolvimento de um Framework

Entenda o que está por trás dos frameworks que usamos todos os dias: princípios, padrões e conceitos fundamentais



Ao final da aula o estudante deverá ser capaz de:

- Compreender o que caracteriza um **framework de software**.
- Identificar os **requisitos fundamentais para criação de um framework**.
- Entender os conceitos de **modularidade** e **extensibilidade**.
- Compreender o conceito de **Inversão de Controle (IoC)**.
- Diferenciar **frameworks de bibliotecas tradicionais**.
- Reconhecer como frameworks modernos implementam esses conceitos.



O que Caracteriza um Framework?

Fundação Reutilizável

Um framework é uma **estrutura pré-construída e genérica** que resolve problemas recorrentes de desenvolvimento. Em vez de recomeçar do zero, você trabalha sobre uma base sólida já testada.

Inversão de Controle

O framework **chama o seu código**, não o contrário. Esse princípio — chamado de IoC (Inversion of Control) — é o que distingue um framework de uma simples biblioteca.

Arquitetura Definida

Frameworks impõem uma **estrutura e padrões arquiteturais** (como MVC ou camadas de serviço), garantindo organização e consistência em todo o projeto.

Em resumo: um framework é um **esqueleto de aplicação** com pontos de extensão onde o desenvolvedor insere a lógica específica do seu domínio.

Requisitos para Criação de Frameworks

Para que um framework seja realmente útil e adotado, ele precisa atender a um conjunto de requisitos essenciais de engenharia de software.

1

Modularidade

O framework deve ser organizado em **módulos independentes e coesos**, permitindo que o desenvolvedor utilize apenas o que precisa, sem carregar funcionalidades desnecessárias.

2

Extensibilidade

Deve ser possível **adicionar novas funcionalidades** sem alterar o código-fonte do framework. O framework define contratos (interfaces, hooks, eventos) que o desenvolvedor implementa.

3

Inversão de Controle (IoC)

O fluxo de execução deve ser **controlado pelo framework**. O desenvolvedor registra seus componentes e o framework os invoca no momento adequado.

4

Documentação e Convenções Claras

Frameworks bem-sucedidos possuem **contratos explícitos**: convenções de nomenclatura, estrutura de pastas e APIs bem documentadas que orientam o desenvolvedor.

5

Testabilidade

A arquitetura do framework deve facilitar a **escrita de testes unitários e de integração**, desacoplando dependências e promovendo injeção de dependência.

Modularidade

Dividir para conquistar — o princípio fundamental da organização

O que é Modularidade?

Modularidade é a propriedade de um sistema ser composto por **módulos independentes, com responsabilidades bem definidas e baixo acoplamento** entre si.

Cada módulo **encapsula** sua lógica interna e expõe apenas uma interface pública para o restante do sistema.

 Princípio SRP (Single Responsibility Principle): cada módulo deve ter **uma única razão para mudar**.

Modularidade em Frameworks

→ Núcleo (Core)

Funcionalidades essenciais e indispensáveis do framework, sempre presentes.

→ Módulos Opcionais

Recursos adicionais (autenticação, ORM, cache) que são incluídos conforme a necessidade do projeto.

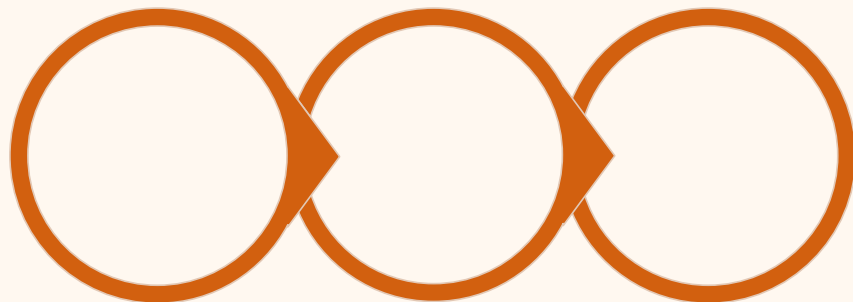
→ Plugins de Terceiros

Extensões criadas pela comunidade que se encaixam na estrutura modular sem modificar o core.

Exemplo prático: O NestJS permite importar módulos de banco de dados, autenticação e filas de forma independente — você só inclui o que o seu projeto precisa.

Extensibilidade

Aberto para extensão, fechado para modificação — o Princípio Aberto/Fechado (OCP)



Um framework extensível define **pontos de extensão explícitos** — interfaces, classes abstratas, hooks e eventos — onde o desenvolvedor insere sua lógica sem precisar alterar o código do framework.

Mecanismos de Extensão

- **Interfaces e classes abstratas:** contratos que o desenvolvedor implementa
- **Hooks e callbacks:** pontos do ciclo de vida onde o código customizado é chamado
- **Eventos e listeners:** sistema de pub/sub para reagir a ações do framework
- **Decorators/Annotations:** metadados que configuram comportamentos sem alterar o fluxo

Benefícios da Extensibilidade

- O framework evolui sem quebrar o código existente
- Desenvolvedores contribuem com plugins sem acesso ao core
- Comportamentos padrão podem ser substituídos quando necessário
- Facilita testes com implementações *mock* das interfaces

Inversão de Controle (IoC)

O conceito que define a essência de um framework

O Fluxo Tradicional

No desenvolvimento **sem framework**, o seu código está no centro: você chama as bibliotecas e define toda a sequência de execução do programa.

⚠ **Sem IoC:** seu código → chama → bibliotecas
O desenvolvedor controla tudo.

Com Inversão de Controle

Com um framework, o **framework assume o controle do fluxo**. Ele chama o seu código nos momentos certos, seguindo seu ciclo de vida interno.

✅ **Com IoC:** framework → chama → seu código
O framework orquestra a execução.

Injeção de Dependência (DI)

A forma mais comum de implementar IoC. Em vez de o seu código instanciar dependências, o **framework injeta as dependências** automaticamente.

```
// SEM IoC (acoplamento forte)
class PedidoService {
    repo = new PedidoRepository(); //
```

Biblioteca vs. Framework

Entendendo a diferença fundamental

A Analogia Perfeita

Biblioteca: Você está no comando. É como ter uma caixa de ferramentas — você escolhe qual ferramenta usar e quando usar.

Framework: O framework está no comando. É como uma linha de montagem — você segue o processo estabelecido e preenche as lacunas.

Comparação Técnica

Característica	Biblioteca	Framework
Controle do fluxo	Desenvolvedor	Framework
Inversão de controle	Não	Sim
Arquitetura definida	Não	Sim
Flexibilidade	Alta	Moderada
Curva de aprendizado	Baixa	Alta
Exemplos	Lodash, Axios	Spring, Django

Perguntas para Reflexão

Reflexão Individual

- Quais dos requisitos estudados (modularidade, extensibilidade, IoC) você considera mais difícil de implementar? Por quê?
- Como a inversão de controle muda a forma como você pensa na estrutura de um projeto?
- Em que situação você preferiria usar uma biblioteca simples em vez de um framework completo?

Para Discussão em Aula

- Ao criar um sistema para um cliente, como você garantiria que ele seja extensível sem abrir mão da simplicidade?
- Modularidade e baixo acoplamento são suficientes para um bom framework? O que mais seria necessário?
- Como a injeção de dependência favorece a escrita de testes automatizados?

Próxima aula: Colocaremos esses conceitos em prática — exploraremos a instalação, configuração e criação do nosso primeiro projeto usando um framework moderno.

OBRIGADO PELA ATENÇÃO!